

---

|                  |                                                    |
|------------------|----------------------------------------------------|
| Workgroup:       | Network Working Group                              |
| Internet-Draft:  | draft-wullink-rpp-multi-party-authorization-00     |
| Published:       | 14 November 2026                                   |
| Intended Status: | Standards Track                                    |
| Expires:         | 18 May 2027                                        |
| Authors:         | M. Wullink      P. Kowalik<br>SIDN Labs      DENIC |

# Multi-Party Authorization for RESTful Provisioning Protocol (RPP)

---

## Abstract

The traditional Registry, Registrar and Registrant (RRR) model for domain name management is evolving to include third-party service providers, such as DNS hosting providers. This document describes a generic multi-party authorization flow that allows registries to securely delegate domain management functions to accredited third-party service providers, while ensuring that the registrant's explicit consent is obtained before any RPP operations, described in [I-D.ietf-rpp-core], are executed.

## TODO

## Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 18 May 2027.

## Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

# Table of Contents

|                                      |    |
|--------------------------------------|----|
| 1. Introduction                      | 4  |
| 2. Terminology                       | 4  |
| 3. Conventions Used in This Document | 4  |
| 4. Architectural Overview            | 5  |
| 4.1. Authorization Request           | 5  |
| 4.2. Authorization Usage             | 7  |
| 4.3. Authorization Management        | 7  |
| 5. Data Objects                      | 7  |
| 5.1. Component Data Objects          | 8  |
| 5.1.1. Public Key Data Object        | 8  |
| 5.1.2. Signature Data Object         | 8  |
| 5.1.3. Approval Data Object          | 8  |
| 5.2. Authorisation Data Object       | 8  |
| 5.2.1. Elements                      | 8  |
| 5.2.2. Usage of elements by parties  | 8  |
| 5.2.3. JSON Schema                   | 9  |
| 5.3. Organization Data Object        | 10 |
| 5.3.1. Approval Redirect URL         | 11 |
| 5.3.2. Return Redirect URL           | 11 |
| 6. Request Processing                | 11 |
| 6.1. Third Party                     | 11 |
| 6.2. Registry                        | 12 |
| 6.3. Registrar                       | 12 |
| 7. Approval Lifecycle                | 12 |
| 8. Request Signing and Verification  | 13 |
| 8.1. Public Keys                     | 13 |
| 8.1.1. JSON Schema                   | 14 |

---

|                                         |    |
|-----------------------------------------|----|
| 8.2. Canonicalization and Signing Input | 14 |
| 8.3. Registry                           | 14 |
| 8.4. Registrar                          | 15 |
| 9. Transaction Types                    | 15 |
| 9.1. Domain Name Server Update          | 15 |
| 9.2. Domain DNSSEC Update               | 16 |
| 9.3. Domain Transfer                    | 16 |
| 10. HTTP                                | 16 |
| 10.1. Endpoints                         | 16 |
| 11. Examples                            | 17 |
| 11.1. Key provisioning                  | 17 |
| 11.2. Redirection URL management        | 18 |
| 11.3. Domain Name Server Update Request | 18 |
| 11.4. Domain DNSSEC Update Request      | 19 |
| 11.5. Domain Transfer Request           | 19 |
| 12. Security Considerations             | 21 |
| 13. Result Codes                        | 22 |
| 13.1. Client Errors                     | 22 |
| 13.2. Server Errors                     | 23 |
| 14. IANA Considerations                 | 23 |
| 15. Internationalization Considerations | 24 |
| 16. Privacy Considerations              | 24 |
| 17. Change History                      | 24 |
| 18. References                          | 24 |
| 18.1. Normative References              | 24 |
| 18.2. Informative References            | 25 |
| Acknowledgements                        | 25 |
| Authors' Addresses                      | 25 |

## 1. Introduction

The generic multi-party authorization flow described in this document allows registries to securely delegate RPP object operations to accredited third-party service providers, while ensuring that the registrar and registrant's explicit consent is obtained before any RPP operations, described in [\[I-D.ietf-rpp-core\]](#), are executed. The registry and the 3rd party MUST have a pre-established trust relationship, how this trust is established is out of scope for this document. The 3rd party does not have a direct trust relationship with the registrar, but the registrar trusts the registry to only accept requests from accredited 3rd parties. How to secure the HTTP endpoints used in the multi-party authorization flow is out of scope for this document, but it is RECOMMENDED that OAuth 2.0 for RPP, as described in [\[I-D.wullink-rpp-oauth2\]](#), be used.

**TODO** what about registries that allow direct registrar access, without a registrar?

## 2. Terminology

In this document the following terminology is used.

URL - A Uniform Resource Locator as defined in [\[RFC3986\]](#).

Resource - An object having a type, data, and possible relationship to other resources, identified by a URL.

RPP client - An HTTP user agent performing an RPP request

RPP server - An HTTP server responsible for processing requests and returning results in any supported media type.

Registry - The authoritative source of truth for registration data, responsible for maintaining the database of shared objects and providing access to it through the RPP server.

Registrar - The entity responsible for managing the registration of objects, such as a domain name, on behalf of registrants, typically interacting with both the registrant and the registry.

Registrant - The individual or organization that has registered an object in the registry database.

3rd party - A service provider that is accredited by the registry to perform domain management operations on behalf of the registrant.

## 3. Conventions Used in This Document

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [\[RFC2119\]](#).

In examples, indentation and white space are provided only to illustrate element relationships and are not REQUIRED features of the protocol.

## 4. Architectural Overview

RPP enables registries to securely delegate domain management functions to accredited third-party service providers, such as DNS hosting providers.

### 4.1. Authorization Request

The authorization of such a delegated operation requires the explicit participation of four parties: the registrant, the third party performing the operation, the registry, and the registrar managing the object resource. The registry and the registrar each apply their own digital signature to the authorization request, the operation **MUST** only be executed once both signatures have been verified.

The diagram in [Figure 1](#) illustrates the multi-party authorization flow:

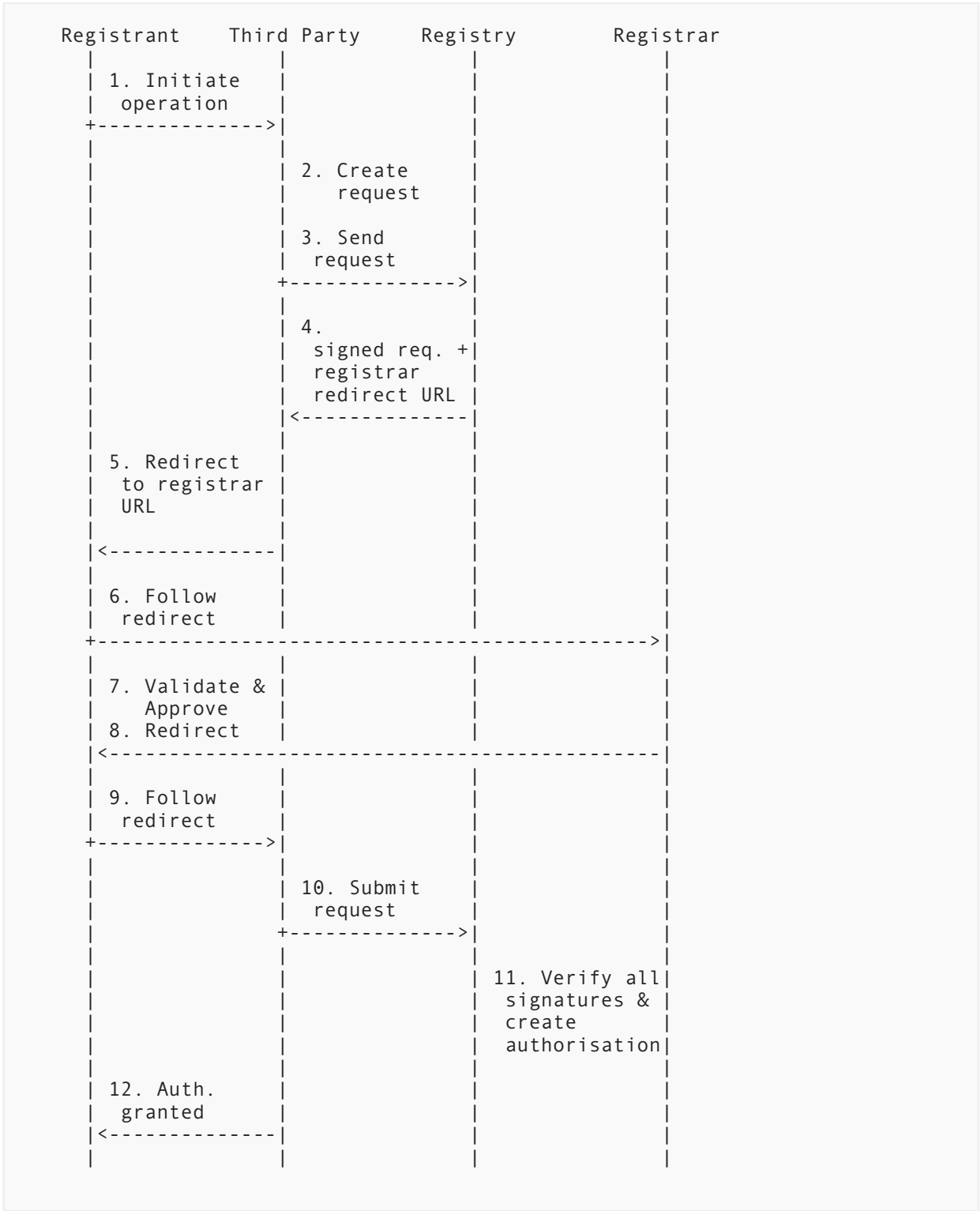


Figure 1: multi-party Authorization Flow

The individual steps in the diagram [Figure 1](#) performed by each party are described below:

1. The client connects to the third party to initiate a domain management operation that requires authorization from the registry and the registrar.
2. The third party's backend constructs a request describing the requested operation, expressing the third party's intent to perform the operation on behalf of the registrant.
3. The third party sends the request to the RPP authorization endpoint.
4. The registry checks if the third party is accredited to act on the object and validates the request, then it signs the request. The registry returns the request, together with the URL of the registrar's web-based consent screen to which the third party must redirect the client's browser for transaction approval.
5. The third party redirects the client's browser to the registrar's consent screen at the URL provided by the registry, conveying the signed request as a parameter of the redirect.
6. The client's browser follows the redirect to the registrar.
7. The registrar validates the signature of the registry, and checks if the request is valid and allowed per the registrar's policies. The registrar then presents the requested operation to the registrant for approval. If the registrant approves, the registrar signs the response with its own signature.
8. The registrar redirects the client's browser back to the third party.
9. The client's browser follows the redirect, delivering the request signed by the registry and the registrar to the third party.
10. The third party submits the authorization request to the registry, including the signatures of both the registry and the registrar.
11. The registry verifies the signatures of the registry itself and the registrar. The registry creates an Authorisation Data Object, which is stored in the registry's database and used to authorize future operations.
12. The third party informs the client that the requested authorisation has been granted.

## 4.2. Authorization Usage

After the multi-party authorization flow is completed, the registry **MUST** use the Authorisation Data Object to authorize an operation request by a 3rd party. If the object in the request of a 3rd party is linked to a valid Authorisation Data Object, the registry **MUST** execute the requested operation and return the result to the 3rd party. If there is no Authorisation Data Object or it is invalid, the registry **MUST** reject the request and return an appropriate error response to the 3rd party.

**TODO** add diagram showing how the Authorisation Data Object is used to authorize an operation request by a 3rd party.

## 4.3. Authorization Management

An authorization granted by the registry **MAY** be terminated by the 3rd party, the registrar or the registry at any time, before the expiration of the Authorisation Data Object.

**TODO** add diagram showing how the Authorisation Data Object is revoked by the registrar or registry or rescinded by the 3rd party.

## 5. Data Objects

**TODO** here or in data objects doc? (now in data objects doc)

## 5.1. Component Data Objects

### 5.1.1. Public Key Data Object

TODO

### 5.1.2. Signature Data Object

TODO

### 5.1.3. Approval Data Object

TODO

## 5.2. Authorisation Data Object

### 5.2.1. Elements

This section describes the data elements of the Authorisation Data Object, as defined in [\[I-D.ietf-rpp-data-objects\]](#).

- `transactionType`: The type of transaction for the authorisation request. MUST be one of the values registered in the "RPP Multi-Party Transaction Types" registry [Table 6](#). Every transaction type is associated with a specific RPP operation, and the data property of the authorisation request MUST contain the RPP request body for that operation.
- `id`: The identifier for the authorisation request.
- `timestamp`: The time the authorisation request was created.
- `expiration`: The time the authorisation request expires. MUST be later than `timestamp`.
- `objectId`: The identifier of the object the authorisation request pertains to.
- `requestorId`: The unique organisation identifier of the requestor.
- `requestorName`: The name of the requestor.
- `approvalUrl`: The URI to which the client should be redirected for multi-party approval. MUST only be used when the referenced organisation is a registrar or reseller supporting multi-party approval.
- `returnUrl`: The URI to which the client should be redirected after multi-party approval at the registrar. MUST only be used when the referenced organisation is a 3rd party supporting multi-party approval.
- `usage`: The usage type for the authorisation request. MUST be one of "single-use" or "multi-use", the default being "single-use".
- `data`: The data associated with the authorisation request, containing the RPP request body for the operation identified by `transactionType`; MUST be a valid RPP Data Object, see [Section 9](#).
- `signatures`: The digital signatures applied to the authorisation request, used to verify its authenticity and integrity, see [Section 8](#).
- `approval`: The approval information for the authorisation request.

### 5.2.2. Usage of elements by parties

The Authorisation Data Object elements are set by different parties involved in the multi-party authorization flow, and read by others as needed to process or verify the transaction.



| Identifier      | Set by              | Read by              |
|-----------------|---------------------|----------------------|
| transactionType | 3rd party           | Registry, Registrar  |
| id              | Registry            | 3rd party, Registrar |
| timestamp       | Registry            | 3rd party, Registrar |
| expiration      | Registry            | 3rd party, Registrar |
| objectId        | 3rd party           | Registry, Registrar  |
| requestorId     | Registry            | Registrar            |
| requestorName   | Registry            | Registrar            |
| approvalUrl     | Registry            | 3rd party            |
| returnUrl       | Registry            | Registrar            |
| usage           | 3rd party           | Registry, Registrar  |
| data            | 3rd party           | Registry, Registrar  |
| signatures      | Registry, Registrar | Registry, Registrar  |
| approval        | Registrar           | Registry, 3rd party  |

Table 1: Party responsible for setting and reading each data element

### 5.2.3. JSON Schema

This section provides normative JSON Schema definitions for the transaction types defined in this document. All schemas use JSON Schema draft 2020-12 [JSON-SCHEMA]. The schema below represents the Authorisation Data Object defined in [I-D.ietf-rpp-data-objects] and described in Section 5.2.1.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/authorisation.create",
  "$defs": {
    "authorisation.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "authorisation" },
        "transactionType": { "type": "string", "enum": ["rpp:mpa-type:domain:ns-update", "rpp:mpa-type:domain:dnssec-update", "rpp:mpa-type:domain:transfer"] },
        "id": { "type": "string" },
        "timestamp": { "type": "string", "format": "date-time" },
        "expiration": { "type": "string", "format": "date-time" },
        "objectId": { "type": "string" },
        "requestorId": { "type": "string" },
        "requestorName": { "type": "string" },
        "approvalUrl": { "type": "string", "format": "uri" },
        "returnUrl": { "type": "string", "format": "uri" },
        "usage": { "type": "string", "enum": ["single-use", "multi-use"] },

```

```

    "data": {
      "description": "The RPP JSON request body for the operation
identified by transactionType"
    },
    "signatures": {
      "type": "array",
      "description": "An ordered list of signatures",
      "items": {
        "type": "object",
        "properties": {
          "keyId": { "type": "string" },
          "signedAt": {
            "type": "string",
            "format": "date-time",
            "description": "The date and time at which this signature was
applied."
          },
          "signature": { "type": "string" }
        },
        "required": ["keyId", "signedAt", "signature"],
        "additionalProperties": false
      },
      "minItems": 0
    },
    "approval": {
      "type": "object",
      "properties": {
        "approved": { "type": "boolean" },
        "approvedBy": { "type": "string" },
        "reason": { "type": "string" },
        "timestamp": { "type": "string", "format": "date-time" }
      },
      "required": ["approved", "approvedBy", "timestamp"],
      "additionalProperties": false
    }
  },
  "required": ["@type", "transactionType", "id", "timestamp",
"expiration", "objectId", "requestorId"]
}
}

```

**TODO** currently using single schema where depending on client , registry or registrar some fields are required and some are optional. Could also define separate schemas for each party.

### 5.3. Organization Data Object

This specification extends the Organization type specified in [\[I-D.ietf-rpp-data-objects\]](#) to include two additional elements: `approvalUrl`, which is a URL that points to the registrar's web-based consent screen, and `returnUrl`, which is a URL to which the client's browser is redirected after the registrant has approved or denied the requested operation. The `approvalUrl` and `returnUrl` **MUST** be valid URLs, and they **MUST** use the HTTPS scheme.

The registry and the registrar **MUST** redirect the client's browser using a 303 See Other response, with the Location header set to the URL of the registrar's consent screen, including base64 encoded representation of the Authorisation Data Object in the approval-state query parameter.

This specification does not define a specific format for the redirection URL, but it is **RECOMMENDED** that the URL be constructed using the following format:

```
https://<hostname>/<path>?approval-state=<base64-encoded-authorisation-data-object>
```

### 5.3.1. Approval Redirect URL

A registrar that supports multi-party authorization **MUST** provide a web-based consent screen to which the 3rd party can redirect the client's browser to obtain the registrant's approval for the requested operation. The registrar **MUST** provide a URL for this consent screen to the registry, which is used by the registry to inform the 3rd party where to redirect the client's browser.

Example registrar approval endpoint URL, with the approval-state query parameter containing the base64 encoded Authorisation Data Object:

```
HTTP/1.1 303 See Other
Location: https://registrar.example/rpp-auth/approve?approval-state=abc123
```

### 5.3.2. Return Redirect URL

A 3rd party that supports multi-party authorization **MUST** provide a URL to which the client's browser is redirected by the registrar, after the registrar and registrant have completed the approval process. The 3rd party **MUST** provide this URL to the registry, which then appends it to the request, so that the registrar can redirect the client's browser back to the 3rd party after the approval process is completed.

Example 3rd party approval endpoint URL, with the approval-state query parameter containing the base64 encoded Authorisation Data Object:

```
HTTP/1.1 303 See Other
Location: https://thirdparty.example/rpp-auth/callback?approval-state=abc123
```

## 6. Request Processing

The request is initiated by the 3rd party, and validated and processed by both the registry and registrar. The request **MUST** be signed by both the registry and registrar.

### 6.1. Third Party

The 3rd party constructs the request, setting the transactionType, objectId, expiration and data properties, and sends the request to the registry.

## 6.2. Registry

The registry, after validation of the request, adds the following properties to the request: `id`, `requestorId`, `requestorName`, `approvalUrl`, `returnUrl`, `timestamp`, `signatures`. The `id` is a unique identifier for the transaction. The `requestorId` is the identifier of the 3rd party that created the request, the `requestorName` is the public name of the requestor to be displayed in the consent screen. The `approvalUrl` is the URL where the request can be approved, the `returnUrl` is the URL to return to after approval, the `timestamp` is the time the request was created, the `expiration` is the time the request expires, the `signatures` contain the cryptographic signatures, the registry MUST add its signature of the request to the `signatures` array, it MUST be the first element in the array.

## 6.3. Registrar

The registrar adds the `approval` object to the request, to indicate whether the registrant and registrar have approved the transfer; at minimum, `approval.approved` and `approval.approvedBy` MUST be set. The registrar also adds its signature and public key identifier to the `signatures` array in the request, it MUST be the second element in the array. The registrar MUST verify the signature of the registry using the public key linked to the public key identifier in the request.

# 7. Approval Lifecycle

The `expiration` property of the Authorisation Data Object indicates the time the approved request expires. The registry and registrar MUST reject any request that has expired, and return an appropriate error response to the 3rd party. The 3rd party MUST ensure that the request is submitted to the registry before it expires.

The registry MAY set a maximum expiration time for requests, and reject any request that exceeds this limit. The registrar MAY also set a maximum expiration time for requests, and reject any request that exceeds this limit.

The `usage` property of the Authorisation Data Object indicates whether the request is a single-use or multi-use request. The registry MUST keep track of the usage of the request, and reject any request that has already been used if it is a single-use request.

The 3rd party may rescind an approval it has previously granted, at any point while the authorisation remains valid, to immediately terminate its ongoing access to the object. The registry MUST check if the authorisation is linked to the 3rd party, and if so, it MUST invalidate the Authorisation Data Object and reject any subsequent RPP operation request that relies on it.

The registrar MAY rescind an approval it has previously granted, at any point while the authorisation remains valid, to immediately terminate the 3rd party's ongoing access to the object. This is particularly relevant for "multi-use" requests, which by design grant the 3rd party continuing access to perform the authorised operation on the object until the request expires or is rescinded.

To rescind an approval, the registrar sends a revocation request to the registry, identifying the Authorisation Data Object by its `id` and signed by the registrar using the same key used to approve the original request. Upon receiving a valid, signed revocation request, the registry MUST immediately invalidate the referenced Authorisation Data Object, regardless of its `expiration` or remaining uses, and MUST reject any subsequent RPP operation request that relies on it.

The registrar MAY rescind an approval, for any object under its management, for any reason, including but not limited to: the registrant revoking consent, a change in the registrant's circumstances, or a violation of the registrar's policies by the 3rd party. The registry MAY also rescind an approval if it determines that the 3rd party is no longer accredited or if it detects suspicious or malicious activity.

The registry MUST inform the 3rd party that the authorisation has been rescinded, so that the 3rd party can stop relying on it and, where applicable, notify its own client.

## 8. Request Signing and Verification

Both the registry and the registrar MUST cryptographically sign, and verify the authenticity and integrity of the requests. The specific algorithms and key management practices are outside the scope of this document, but it is RECOMMENDED that parties follow best practices for cryptographic security.

Each party MUST sign only the data that it is responsible for, and MUST include its public key identifier in the request, so that other parties can verify the signature using the corresponding public key.

### 8.1. Public Keys

A registrar that wants to participate in multi-party authorization MUST provide at least one public key to the registry. Each key is identified by a unique key identifier, which is used to verify signatures applied by the corresponding private key.

The RPP server MUST provide the list of its own public keys and their identifiers to the 3rd party and the registrar, using the discovery document available at the well-known endpoint as described in [\[I-D.ietf-rpp-core\]](#). The Discovery document MUST be extended to include the additional public keys and their identifiers, using the field name `registryKeys`, which is an array of JSON Web Key (JWK) objects [\[RFC7517\]](#), the `kid` field of each JWK object MUST be used as the key identifier.

The `use` field of each JWK object MUST be set to `sig`, indicating that the key is used for digital signatures. The `alg` field of each JWK object MUST be set to the algorithm used to sign requests, such as `RS256` for RSA signatures using SHA-256.

The following example shows a discovery document with 2 registry public keys:

```
{
  "registryKeys": [
    {
      "kty": "RSA",
      "kid": "registry-key-1",
      "use": "sig",
      "alg": "RS256",
      "n": "0vx7agoebGcQSuuPiLJXZptN9nndrQmbXEps2aiAFbWhM78LhWx4cbb....",
      "e": "AQAB"
    },
    {
      "kty": "EC",
      "kid": "registry-key-2",
      "use": "sig",
      "alg": "ES256",
      "crv": "P-256",
      "x": "f830J3D2xF1Bg8vub9tLe1gHMzV76e8Tus9uPHvRVEU",
      "y": "x_FEzRu9m36HLN_tue659LNpXW6pCyStikYjKIWI5a0"
    }
  ]
}
```

### 8.1.1. JSON Schema

TODO

## 8.2. Canonicalization and Signing Input

Signers **MUST NOT** construct the signature input by naively concatenating field values as strings. Ad hoc concatenation is ambiguous: for example, concatenating the values "ab" and "c" produces the same string as concatenating "a" and "bc", and differences in whitespace, member ordering, or numeric formatting between a signer's and a verifier's JSON serializer can cause a correctly-signed request to fail verification.

Instead, the signature input **MUST** be derived by applying the JSON Canonicalization Scheme (JCS) [[RFC8785](#)] to a JSON object containing exactly the fields covered by that signature, encoded as UTF-8 octets. JCS produces a single, deterministic byte string for a given set of field values, independent of the original member order or formatting, making the signature input reproducible by any conforming verifier.

A signature **MUST NOT** cover the signature or key identifier fields of any other party. Chaining a later signature over earlier signatures would add no additional protection. Each party **MUST** include its own public key identifier alongside its signature, so that other parties can verify the signature using the corresponding public key.

## 8.3. Registry

The registry **MUST** verify the transaction request, and copy the document to a response document while adding additional transaction related properties, and finally sign the response document using its private key. The signature **MUST** be included in the response object.

The registry signs the JCS-canonicalized form of:

- transactionType
- id

- timestamp
- expiration
- objectId
- requestorId
- approvalUrl
- returnUrl
- data

The registry inserts the signature as an object in the `signatures` array.

## 8.4. Registrar

The registrar MUST verify the signature of the registry using the public key linked to the public key identifier found in the first element of the `signatures` array, and copy the request to a response document while adding additional transaction related properties, and finally sign the response using its private key. The signature MUST be included in the response object.

The registrar signs the JCS-canonicalized form of all properties also signed by the registry, but it MUST include the following additional properties in the signature input:

- approval
- ...

The registrar inserts the signature as an object in the `signatures` array.

## 9. Transaction Types

A transaction type allows access to specific RPP operations. This specification defines three distinct transaction types. Future specifications may define additional transaction types, which MUST be registered with IANA as described in [Section 14](#).

- `rpp:mpa-type:domain:ns-update`
- `rpp:mpa-type:domain:dnssec-update`
- `rpp:mpa-type:domain:transfer`

This specification does not define a separate payload format for the RPP operation being authorized. Instead, the `data` property of a transaction request MUST contain the same JSON request message that would normally be sent directly to the RPP server for the corresponding operation, using the mapping rules and object representations defined in [\[I-D.ietf-rpp-json\]](#).

### 9.1. Domain Name Server Update

The `rpp:mpa-type:domain:ns-update` transaction type is used to update the delegation details for a domain name, by replacing the set of Host Data Objects referenced in the domain's `nameservers` property with a new set of Host Data Objects. The `objectId` property MUST contain the domain name of the target Domain Name Object.

The `data` property MUST contain a valid RPP JSON partial update request body (a JSON Patch document, as defined in the Partial Update rules of [\[I-D.ietf-rpp-json\]](#)), consisting of one `replace` operation for each existing Host Data Object being replaced. Each operation's `path` MUST be `/nameservers`, its `match` property MUST identify the existing Host Data Object being replaced by its `hostName`, and its `value` property MUST contain the new Host Data Object.

## TODO

### 9.2. Domain DNSSEC Update

The `rpp:mpa-type:domain:dnssec-update` transaction type is used to update the DNSSEC related records of a domain name, by replacing the set of DS or DNSKEY records referenced in the domain's `dns` property with a new set of DS or DNSKEY records. The `objectId` property MUST contain the domain name of the target Domain Name Object.

The `data` property MUST contain a valid RPP JSON partial update request body (a JSON Patch document, as defined in the Partial Update rules of [I-D.ietf-rpp-json]).

## TODO

### 9.3. Domain Transfer

The `rpp:mpa-type:domain:transfer` transaction type is used to transfer the sponsorship of a domain to another registrar. The `objectId` property MUST contain the domain name of the target Domain Name Object, and the `data` property MUST contain a valid Transfer Process Object create request body, as defined in [I-D.ietf-rpp-json], for the transfer operation described in [I-D.ietf-rpp-core].

## TODO

## 10. HTTP

### 10.1. Endpoints

The following non normative table lists RPP endpoints related to Authorisation Objects, each derived by applying the rules defined in section "HTTP Mapping Rules" in [I-D.ietf-rpp-core].

| Operation             | HTTP Method | URL path                                                |
|-----------------------|-------------|---------------------------------------------------------|
| Authorisation: create | "POST"      | <code>"/{collection}/{id}/authorisations"</code>        |
| Authorisation: read   | "GET"       | <code>"/{collection}/{id}/authorisations/latest"</code> |
| Authorisation: read   | "GET"       | <code>"/{collection}/{id}/authorisations/{id}"</code>   |
| Authorisation: delete | "DELETE"    | <code>"/{collection}/{id}/authorisations/{id}"</code>   |
| Authorisation: list   | "GET"       | <code>"/{collection}/{id}/authorisations"</code>        |

Table 2: Authorisation Endpoints {#tbl-authorisation-endpoints} {#tbl-authorisation-process-endpoints}

The following non normative table lists RPP endpoints related to authorisation processes, each derived by applying the rules defined in section "HTTP Mapping Rules" in [I-D.ietf-rpp-core].



| Operation                     | HTTP Method | URL path                                                     |
|-------------------------------|-------------|--------------------------------------------------------------|
| Authorisation Process: create | "POST"      | "/{collection}/{id}/processes/authorisationProcesses"        |
| Authorisation Process: read   | "GET"       | "/{collection}/{id}/processes/authorisationProcesses/latest" |
| Authorisation Process: read   | "GET"       | "/{collection}/{id}/processes/authorisationProcesses/{id}"   |
| Authorisation Process: list   | "GET"       | "/{collection}/{id}/processes/authorisationProcesses"        |

Table 3: Authorisation Process Endpoints

**TODO**

## 11. Examples

### 11.1. Key provisioning

A registrar supporting multi-party authorization MUST provide at least 1 public key at the registry. Public keys are provisioned by adding a Public Key Object to the `publicKeys` property of the Organisation Data Object [I-D.ietf-rpp-data-objects], using an RPP JSON partial update (JSON Patch, as defined in the Partial Update rules of [I-D.ietf-rpp-json]). Since `publicKeys` is a `DictionaryComposition[Public Key Object]`, adding a new key MUST be done using an `add` operation whose path targets the new key identifier directly; the `match` property is not used, as it only applies to array-valued properties.

The following example shows a partial update request adding an RSA public key with identifier `registrar-key-3` to the `publicKeys` property of the Organisation Data Object with id `ORG-12345`:

```

PATCH /organisations/me HTTP/1.1
Authorization: Bearer <access-token>
Content-Type: application/json
[
  {
    "op": "add",
    "path": "/publicKeys",
    "value": {
      "registrar-key-3": {
        "@type": "publicKey",
        "type": "RSA",
        "alg": "RS256",
        "n": "0vx7agoebGcQSuuPiLJXZptN9nndrQmbXEps2aiAFbWhM78LhWx4cbb...",
        "e": "AQAB"
      }
    }
  }
]

```

**TODO** do we want to define "me" as a special identifier for the org of the current loggedin user, makes things easier for clients, will need to update core doc.

## 11.2. Redirection URL management

The registrar's approvalUrl and the 3rd party's returnUrl of their respective Organisation Data Object MUST be set, and MUST be valid URLs. Updating these URLs is done using an RPP JSON partial update of the Organisation Data Object, using a replace operation for each URL. The following example shows a partial update request to set the approvalUrl of the registrar's Organisation Data Object with id ORG-12345:

```
PATCH /organisations/ORG-12345 HTTP/1.1
Authorization: Bearer <access-token>
Content-Type: application/json
[
  {
    "op": "replace",
    "path": "/approvalUrl",
    "value": "https://registrar.example/consent"
  }
]
```

**TODO**

## 11.3. Domain Name Server Update Request

Example 3rd party request to registry to replace the nameservers of a domain object with a new set of Host Data Objects. The data property contains an RPP JSON partial update (JSON Patch) request body as defined in [I-D.ietf-rpp-json], with one replace operation for the existing nameservers. The example below shows a request to update the domain test1.example, replacing the existing nameservers with ns1.dns.example and ns2.dns.example:

```
{
  "@type": "authorisation",
  "transactionType": "rpp:mpa-type:domain:ns-update",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "data": [
    {
      "op": "replace",
      "path": "/nameservers",
      "value": [
        { "@type": "host", "hostName": "ns1.dns.example" },
        { "@type": "host", "hostName": "ns2.dns.example" }
      ]
    }
  ]
}
```

Example Registry Response: **TODO**

Example Registrar Response: **TODO**

## 11.4. Domain DNSSEC Update Request

Example 3rd party request to registry to replace the DNSSEC DS records of a domain object with a new set of DS records. The data property contains an RPP JSON partial update (JSON Patch) request body as defined in [\[I-D.ietf-rpp-json\]](#), with one replace operation for the existing DS records. The example below shows a request to update the domain test1.example, replacing the existing DS records with a new DS record with key tag 12345:

```
{
  "@type": "authorisation",
  "transactionType": "rpp:mpa-type:domain:dnssec-update",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "data": [
    {
      "op": "replace",
      "path": "/dns",
      "value": {
        "@type": "dnsData",
        "records": [
          { "@type": "dnsRecord",
            "name": "@",
            "type": "ds",
            "rdata": { "keyTag": 12345,
                      "algorithm": 8,
                      "digestType": 2,
                      "digest": "ABCDEF1234567890"
                    }
          }
        ]
      }
    ]
  ]
}
```

Example Registry Response: **TODO**

Example Registrar Response: **TODO**

## 11.5. Domain Transfer Request

Example 3rd party request to registry to transfer the management of a domain object. The data property contains a Transfer Process Object as defined in [\[I-D.ietf-rpp-json\]](#):

```
{
  "@type": "authorisation",
  "transactionType": "rpp:mpa-type:domain:transfer",
  "id": "TR-12345",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "data": {
    "@type": "transferProcess",
    "transferDir": "pull"
  }
}
```

Example registry response to the 3rd party to transfer the management of a domain object, signed by the registry, this response is then sent to the losing registrar, by the 3rd party, to request the registrar and registrant's approval for the transfer.

The registry adds the following properties to the request: `id`, `requestorId`, `requestorName`, and `signatures`. The `id` is a unique identifier for the transaction, the `requestorId` is the identifier of the 3rd party that created the request, the `requestorName` is the public name of the requestor to be displayed in the consent screen, and the `signatures` array contains the registry's signature and public key identifier, as the first element in the array.

```
{
  "@type": "authorisation",
  "transactionType": "rpp:mpa-type:domain:transfer",
  "id": "TR-12345",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "requestorId": "ORG-3RDPARTY-1",
  "requestorName": "Example Registrar",
  "data": {
    "@type": "transferProcess",
    "transferDir": "pull"
  },
  "signatures": [
    {
      "keyId": "my-registry-key-1",
      "signedAt": "2027-06-01T12:00:00Z",
      "signature": "yyyyy"
    }
  ]
}
```

Example response from losing registrar, signed by the registrar and the approval object contains the result of the approval process, to indicate whether the registrant and registrar have approved the transfer. The registrar also adds its signature and public key identifier to the `signatures` array in the response.

This response is sent to the registry to request execution of the transfer:

```
{
  "@type": "authorisation",
  "transactionType": "rpp:mpa-type:domain:transfer",
  "id": "TR-12345",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "requestorId": "ORG-3RDPARTY-1",
  "requestorName": "Example Registrar",
  "approval": {
    "approved": true,
    "approvedBy": "registrar-admin@example-registrar.example",
    "timestamp": "2027-06-01T12:05:00Z"
  },
  "data": {
    "@type": "transferProcess",
    "transferDir": "pull"
  },
  "signatures": [
    {
      "keyId": "my-registry-key-1",
      "signedAt": "2027-06-01T12:00:00Z",
      "signature": "yyyyy"
    },
    {
      "keyId": "my-registrar-key-1",
      "signedAt": "2027-06-01T12:05:00Z",
      "signature": "zzzzz"
    }
  ]
}
```

This response is used by the 3rd party to submit the transfer request to the registry. The registry verifies the signatures of the registry and registrar, and confirms that approval.approved is set to true. If both signatures are valid and the approval is granted, the registry **MUST** create an authorization object, allowing the transfer operation to be executed, and return a success response to the 3rd party. The 3rd party can then submit the transfer request to the registry, which will execute the transfer operation.

## 12. Security Considerations

The registrant's explicit consent is required for any RPP operation to be executed. The registrant's consent is obtained through the registrar's web-based consent screen, which is presented to the registrant by the registrar. The registrar **MUST** ensure that the registrant's consent is obtained before any RPP operation is executed.

The registry **MUST** ensure that the registrant is the owner of the domain name for which the RPP operation is requested, and that the registrant's consent is obtained before any RPP operation is executed.

The registry **MUST** ensure that the registrar is the current sponsoring registrar of record for the domain name for which the RPP operation is requested, the sponsoring registrar may have changed between the time the request is created and the time the RPP operation, using the authorization request, is executed.

The registry **MUST** check if the authorization request used to request an RPP operation is correctly signed by the registry itself and the registrar, and that the request is valid and not expired. The registry **MUST** reject any request with a missing or invalid signature, or that is invalid or expired. The Registry **MUST** check that the authorization request is used only once, when the use property is set to `single-use`. The registry **MUST** ensure that the authorization request is used only for the specific RPP operation for which it was created, and that the request is not used to authorize any other operation.

Signature verification **MUST** be performed locally by each verifying party using the public key of the signer, rather than by querying the signer's system at verification time. In particular, the registrar **MUST** verify the registry's signature (and the registry's verification of the 3rd party's signature) using the registry's public key, and **MUST NOT** rely on a live call back to the registry to confirm that a signature is valid. The registry already re-verifies all signatures, when the fully-signed request is submitted for execution (see [Section 4](#)).

This document does not specify any authorization and authentication mechanisms for the 3rd party, registry, and registrar to secure the HTTP endpoints used to exchange requests and responses. It is **RECOMMENDED** to use OAuth 2.0 for RPP, as described in [[I-D.wullink-rpp-oauth2](#)].

The cryptographic algorithms and key management practices used to sign and verify requests are outside the scope of this document, but it is **RECOMMENDED** that parties follow best practices for cryptographic security.

## TODO

# 13. Result Codes

This specification defines new RPP result codes for multi-party authorization, the new result codes follow the result code classes defined in [[I-D.ietf-rpp-core](#)]: 14xxx for client errors and 15xxx for server errors. As described in [[I-D.ietf-rpp-core](#)].

## 13.1. Client Errors

The following client error result codes (class 14xxx) are defined in this document:

| RPP Result Code | HTTP Status Code | Description                                                                                                                                           |
|-----------------|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 14001           | 400 Bad Request  | The <code>transactionType</code> is missing, unknown, or not registered in the "RPP Multi-Party Transaction Types" registry <a href="#">Table 6</a> . |
| 14002           | 400 Bad Request  | The data property does not conform to the RPP JSON request body expected for the given <code>transactionType</code> .                                 |
| 14003           | 404 Not Found    | The <code>objectId</code> does not identify an existing object.                                                                                       |
| 14004           | 403 Forbidden    | One or more required signatures are missing or fail cryptographic verification.                                                                       |
| 14005           | 400 Bad Request  | A <code>keyId</code> referenced in the request does not match any public key currently provisioned by the identified party.                           |

| RPP Result Code | HTTP Status Code | Description                                                                                                  |
|-----------------|------------------|--------------------------------------------------------------------------------------------------------------|
| 14006           | 403 Forbidden    | The 3rd party is not accredited by the registry to perform the requested operation on the referenced object. |
| 14007           | 403 Forbidden    | The identified registrar is not the current sponsoring registrar of record for the referenced object.        |
| 14008           | 400 Bad Request  | The expiration timestamp of the request has already passed.                                                  |
| 14009           | 409 Conflict     | The id of the request has already been used by a previously completed or in-progress transaction.            |
| 14010           | 403 Forbidden    | The registrant did not approve the requested operation at the registrar's consent screen.                    |

Table 4: RPP multi-party authorization client error result codes

**TODO** complete the table with additional result codes

## 13.2. Server Errors

The following server error result codes (class 15xxx) are defined in this document:

| RPP Result Code | HTTP Status Code          | Description                                                               |
|-----------------|---------------------------|---------------------------------------------------------------------------|
| 15001           | 500 Internal Server Error | The registry was unable to sign the authorization request.                |
| 15002           | 500 Internal Server Error | The registry was unable to retrieve or validate a registrar's public key. |

Table 5: RPP multi-party authorization server error result codes

**TODO** complete the table with additional result codes

## 14. IANA Considerations

IANA is requested to register the result codes defined in [Table 4](#) and [Table 5](#) in the "RPP Result codes" registry defined in [\[I-D.ietf-rpp-core\]](#).

IANA is requested to create a new registry for RPP multi-party transaction type identifiers, these identifiers are used to identify the type of RPP transaction used by 3rd parties to request authorization from the registry and registrar. The registry is to be called "RPP Multi-Party Transaction Types" and is to be maintained by the IETF RPP working group.

Name of the registry: RPP Multi-Party Transaction Types  
Registry group: RESTful Provisioning Protocol (RPP)  
Registration procedure: Expert Review

The following transaction types are defined in this document:

| Identifier                        | Description                                   |
|-----------------------------------|-----------------------------------------------|
| rpp:mpa-type:domain:ns-update     | Update DNS hosting (nameservers) for a domain |
| rpp:mpa-type:domain:dnssec-update | Update DNSSEC DS records for a domain         |
| rpp:mpa-type:domain:transfer      | Transfer a domain to another registrar        |

Table 6: RPP multi-party authorization transaction types

TODO

## 15. Internationalization Considerations

TODO

## 16. Privacy Considerations

The registrant personally identifiable information (PII) is not included in the multi-party authorization flow, except for the registrant's explicit consent obtained through the registrar's web-based consent screen.

TODO

## 17. Change History

TODO

## 18. References

### 18.1. Normative References

[I-D.ietf-rpp-core] Wullink, M. and P. Kowalik, "RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-core-00, 6 July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-rpp-core-00>>.

[I-D.ietf-rpp-data-objects] Kowalik, P. and M. Wullink, "RESTful Provisioning Protocol (RPP) Data Objects", Work in Progress, Internet-Draft, draft-ietf-rpp-data-objects-01, 3 July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-rpp-data-objects-01>>.

[I-D.ietf-rpp-json] Wullink, M. and P. Kowalik, "JSON for RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-json-00, 6 July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-rpp-json-00>>.

[I-D.wullink-rpp-oauth2] Wullink, M. and P. Kowalik, "OAuth 2.0 for RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-wullink-rpp-oauth2-00, 6 July 2026, <<https://datatracker.ietf.org/doc/html/draft-wullink-rpp-oauth2-00>>.



- [RFC2119]** Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/info/rfc2119>>.
- [RFC3986]** Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <<https://www.rfc-editor.org/info/rfc3986>>.
- [RFC7517]** Jones, M., "JSON Web Key (JWK)", RFC 7517, DOI 10.17487/RFC7517, May 2015, <<https://www.rfc-editor.org/info/rfc7517>>.
- [RFC8785]** Rundgren, A., Jordan, B., and S. Erdtman, "JSON Canonicalization Scheme (JCS)", RFC 8785, DOI 10.17487/RFC8785, June 2020, <<https://www.rfc-editor.org/info/rfc8785>>.

## 18.2. Informative References

- [JSON-SCHEMA]** JSON Schema, "JSON Schema: A vocabulary for describing JSON Data", 2020, <<https://json-schema.org/draft/2020-12/json-schema-core>>.

## Acknowledgements

TODO

## Authors' Addresses

### Maarten Wullink

SIDN Labs

Email: [maarten.wullink@sidn.nl](mailto:maarten.wullink@sidn.nl)

URI: <https://sidn.nl/>

### Pawel Kowalik

DENIC

Email: [pawel.kowalik@denic.de](mailto:pawel.kowalik@denic.de)

URI: <https://denic.de/>